UNIT 2 – CLOUD SECURITY ARCHITECTURE (CSA)

IAM & Least Privilege Enforcement

Name: Supriya Gawali

Prn No.: 2124UCSF1019

Subject: Cloud Security Architecture (CSA) : Unit 2

Experiment: IAM & Least Privilege Enforcement

---

1. Aim

To implement a secure AWS Identity and Access Management (IAM) structure using the Principle of Least Privilege, enforce Multi-Factor Authentication (MFA) for administrative access, and create an IAM Role that allows an EC2 instance to access Amazon S3 without using hardcoded credentials.

---

2. Objectives

- To understand AWS IAM and access management.
- To create separate IAM groups for different responsibilities.
- To implement the Principle of Least Privilege.
- To create and attach custom IAM policies.
- To configure MFA for administrative access.
- To create an IAM Role for an EC2 instance.
- To provide secure EC2-to-S3 access without hardcoded credentials.
- To understand the security benefits of role-based access control.

---

3. Requirements

- AWS Account

- AWS Management Console
- IAM Service
- Amazon EC2
- Amazon S3
- Authenticator application for MFA
- Internet connection

---

4. Theory

AWS IAM

AWS Identity and Access Management (IAM) is a service that enables secure control of access to AWS resources. IAM provides management of users, groups, roles, policies, and permissions.

Principle of Least Privilege

The Principle of Least Privilege means providing a user or service only the permissions necessary to perform its required tasks.

For example, administrators may require extensive permissions, while auditors may only need read-only access.

IAM Role

An IAM Role provides temporary permissions to AWS services or resources. An EC2 instance can use an IAM Role to access S3 without storing permanent AWS access keys on the instance.

Multi-Factor Authentication

MFA adds an additional authentication factor to an account. This provides additional protection for sensitive administrative access.

---

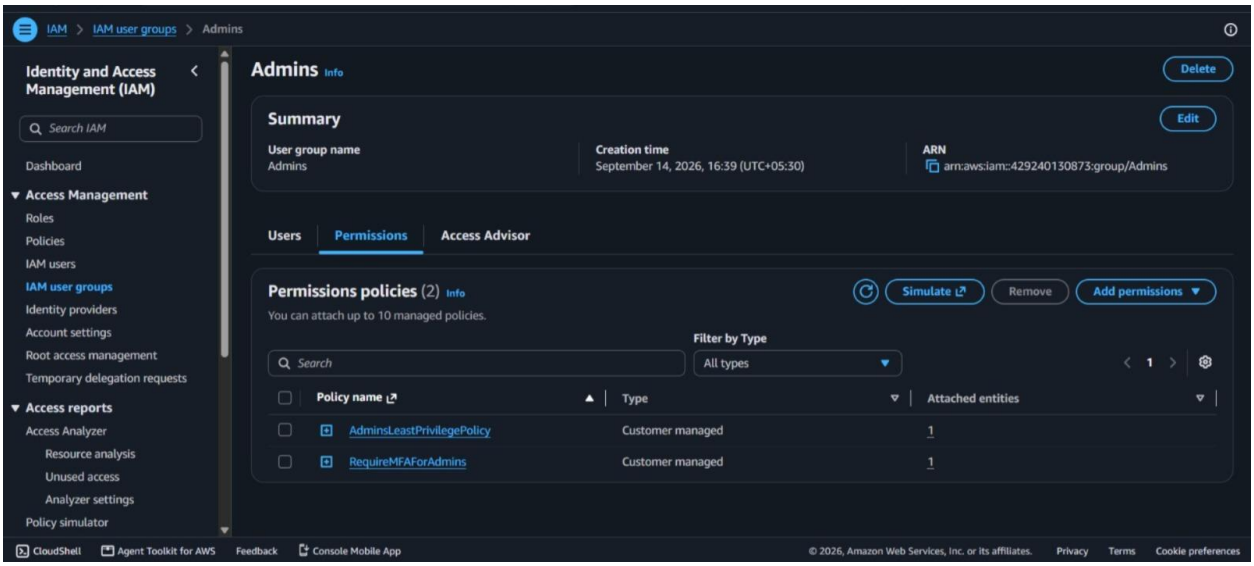
5. Procedure

Step 1: Open AWS IAM

1. Log in to the AWS Management Console.

2. Search for **IAM.**
3. Open the IAM service.
4. Navigate to the IAM dashboard.

Screenshot 1 – IAM Dashboard



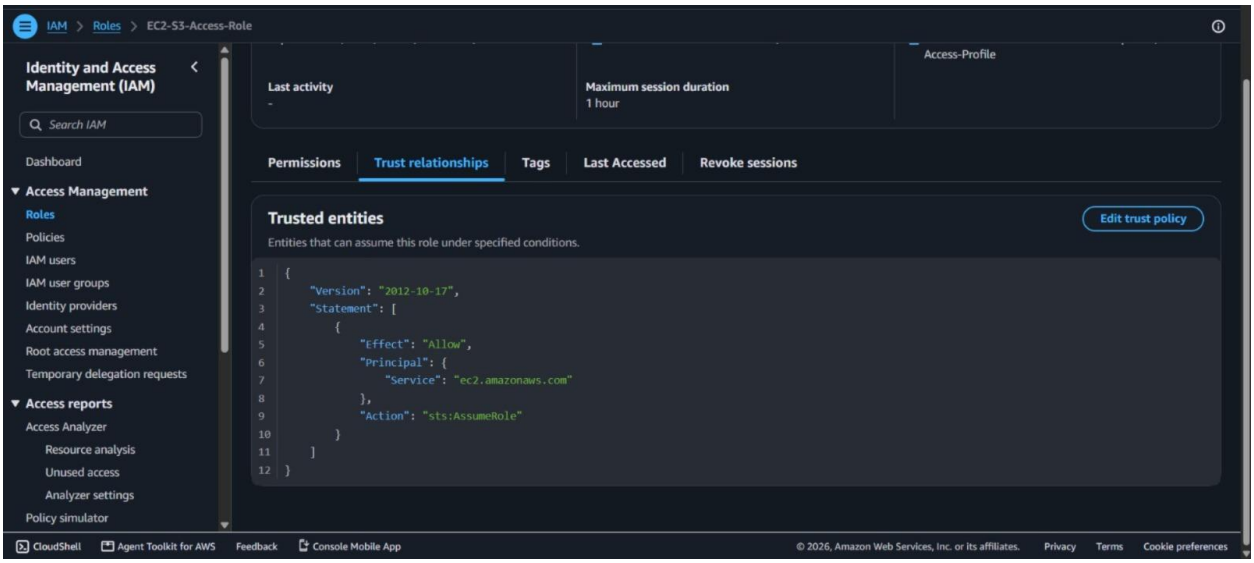
Step 2: Create Admins Group

An IAM group named **Admins** was created for administrative users.

The Admins group was configured with the required administrative permissions.

The purpose of this group is to manage AWS resources and IAM configurations.

Screenshot 2 – Admins Group



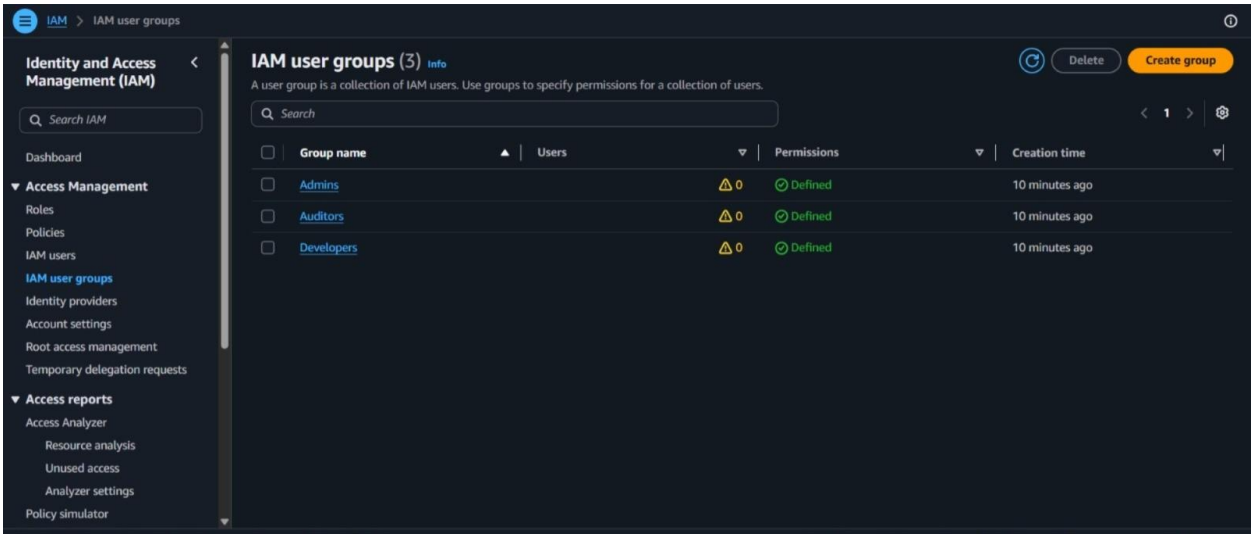
Step 3: Create Developers Group

An IAM group named **Developers** was created.

The group was assigned a custom policy containing only the permissions required for development activities.

This follows the Principle of Least Privilege by avoiding unnecessary administrative permissions.

Screenshot 3 – Developers Group

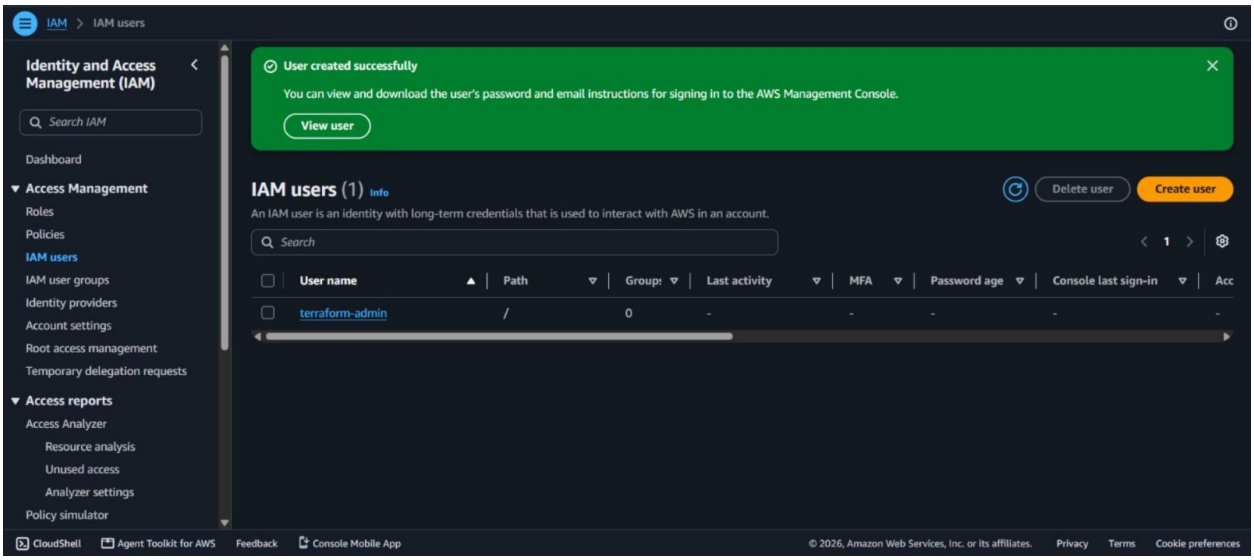


Step 4: Create Auditors Group

An IAM group named **Auditors** was created.

The group was assigned appropriate read-only permissions so that auditors can inspect AWS resources without modifying or deleting them.

Screenshot 4 – Auditors Group



---

6. Custom IAM Policies

Custom IAM policies were created according to the responsibilities of each group.

Admin Policy

The Admin group was provided with administrative permissions required to manage AWS resources.

Developer Policy

The Developer group was provided with limited permissions required for development activities.

Auditor Policy

The Auditor group was provided with read-only permissions for monitoring and auditing purposes.

This separation of permissions helps implement the Principle of Least Privilege.

Screenshot 5 – Custom IAM Policies

```
aws_iam_group.developers: Creation complete after 1s [id=Developers]
aws_iam_policy.admins_policy: Creation complete after 1s [id=arn:aws:iam::429240130873:policy/AdminsLeastPrivilegePolicy]
aws_iam_policy.require_mfa: Creation complete after 1s [id=arn:aws:iam::429240130873:policy/RequireMFAForAdmins]
aws_iam_group_policy_attachment.admins_attach: Creating...
aws_iam_group_policy_attachment.admins_mfa_attach: Creating...
aws_iam_policy.auditors_policy: Creation complete after 1s [id=arn:aws:iam::429240130873:policy/AuditorsReadOnlyPolicy]
aws_iam_group_policy_attachment.auditors_attach: Creating...
aws_iam_role.ec2_s3_role: Creation complete after 2s [id=EC2-S3-Access-Role]
aws_iam_instance_profile.ec2_s3_profile: Creating...
aws_iam_group_policy_attachment.admins_attach: Creation complete after 1s [id=Admins-2026091411094158890000001]
aws_iam_group_policy_attachment.admins_mfa_attach: Creation complete after 1s [id=Admins-2026091411094159400000002]
aws_iam_group_policy_attachment.auditors_attach: Creation complete after 1s [id=Auditors-2026091411094170390000003]
aws_s3_bucket.app_bucket: Creation complete after 4s [id=kaveri-iam-unit2-demo]
aws_s3_bucket_public_access_block.app_bucket: Creating...
data.aws_iam_policy_document.ec2_s3_access: Reading...
data.aws_iam_policy_document.developers_policy: Reading...
data.aws_iam_policy_document.ec2_s3_access: Read complete after 0s [id=2869976164]
data.aws_iam_policy_document.developers_policy: Read complete after 0s [id=1385974031]
aws_iam_policy.ec2_s3_access: Creating...
aws_iam_policy.developers_policy: Creating...
aws_s3_bucket_public_access_block.app_bucket: Creation complete after 0s [id=kaveri-iam-unit2-demo]
aws_iam_policy.ec2_s3_access: Creation complete after 1s [id=arn:aws:iam::429240130873:policy/EC2-S3-Scoped-Access]
aws_iam_policy.developers_policy: Creation complete after 1s [id=arn:aws:iam::429240130873:policy/DevelopersLeastPrivilegePolicy]
aws_iam_role_policy_attachment.ec2_s3_attach: Creating...
aws_iam_group_policy_attachment.developers_attach: Creating...
aws_iam_group_policy_attachment.developers_attach: Creation complete after 0s [id=Developers-2026091411094481050000004]
aws_iam_role_policy_attachment.ec2_s3_attach: Creation complete after 0s [id=EC2-S3-Access-Role-2026091411094483810000005]
aws_iam_instance_profile.ec2_s3_profile: Creation complete after 7s [id=EC2-S3-Access-Profile]

Apply complete! Resources: 17 added, 0 changed, 0 destroyed.

Outputs:

admins_group_name = "Admins"
auditors_group_name = "Auditors"
developers_group_name = "Developers"
ec2_instance_profile_name = "EC2-S3-Access-Profile"
```

---

7. Configure MFA for Administrative Access

MFA was configured to provide an additional security layer for administrative access.

The following steps were performed:

1. Open the IAM user associated with administrative access.
2. Open **Security Credentials.**
3. Select **Assign MFA Device.**
4. Select the required MFA method.
5. Configure the authenticator application.
6. Enter the verification code.
7. Complete the MFA configuration.

MFA helps protect administrative accounts even if the primary password is compromised.

```
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

---

8. Create IAM Role for EC2

An IAM Role was created to allow an EC2 instance to securely access Amazon S3.

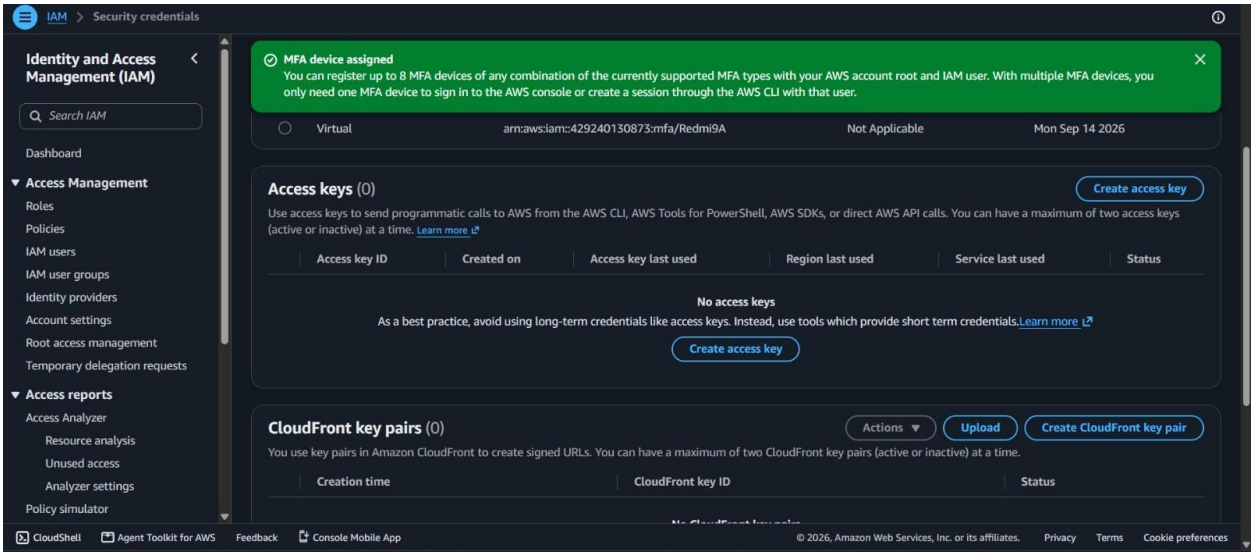
The role was configured with:

Role Name: EC2-S3-Access-Role

The trust relationship was configured for **the EC2 service**, and the required S3 permissions were attached to the role.

Using an IAM Role avoids the need to store permanent AWS access keys on the EC2 instance.

Screenshot 7 – EC2-S3 IAM Role



---

9. Attach IAM Role to EC2

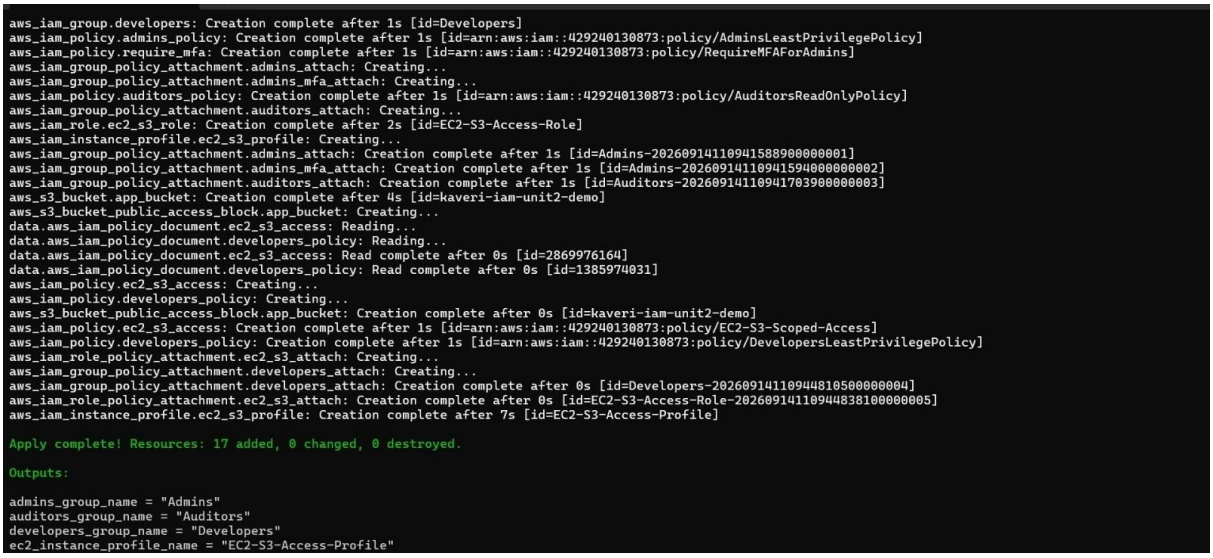
The created IAM Role was associated with the EC2 instance.

The following procedure was performed:

1. Open the **EC2 Console**.
2. Select the required EC2 instance.
3. Choose **Actions.**
4. Select **Security.**
5. Select **Modify IAM Role.**
6. Select EC2-S3-Access-Role.
7. Save the configuration.

The EC2 instance can now use the permissions provided by the IAM Role to access S3.

Screenshot 8 – IAM Role Attached to EC2



---

10. IAM Architecture

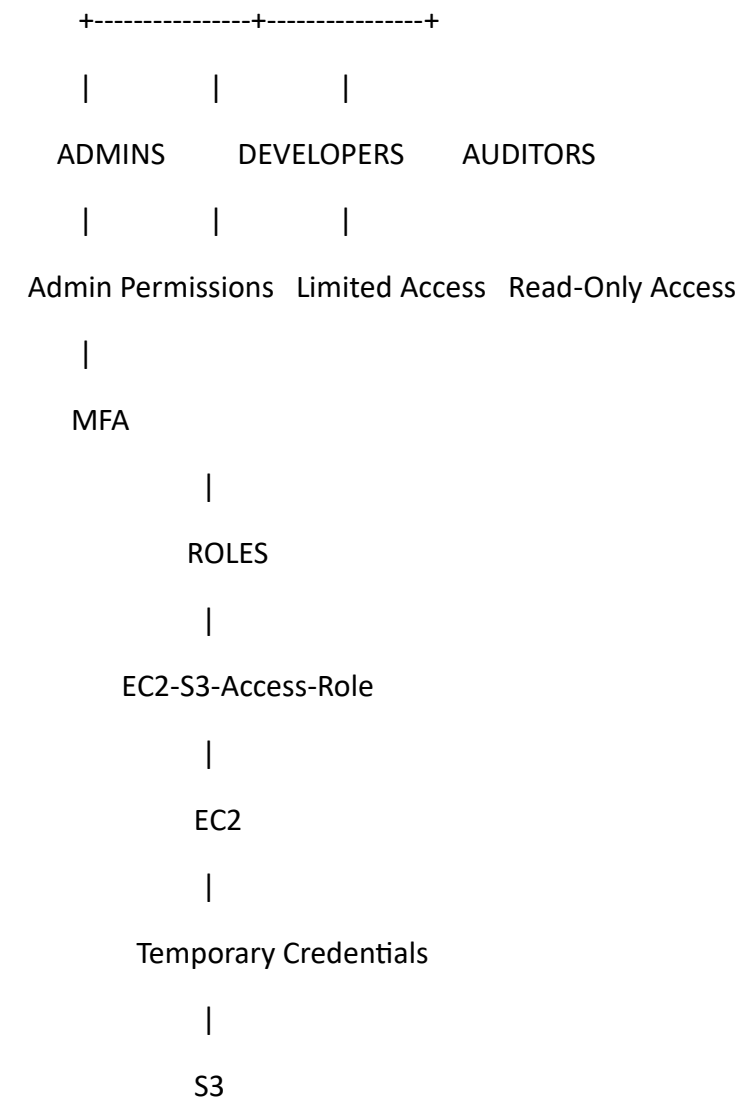
The implemented IAM structure can be represented as:

AWS ACCOUNT

|

IAM

|



---

11. Security Benefits

The implemented IAM structure provides the following security benefits:

1. Least Privilege

Users receive only the permissions necessary for their responsibilities.

2. Separation of Duties

Administrative, development, and auditing responsibilities are separated into different groups.

3. MFA Protection

MFA provides an additional layer of authentication for administrative access.

4. Secure EC2 Access

The EC2 instance accesses S3 using an IAM Role rather than hardcoded access keys.

5. Reduced Security Risk

Limiting unnecessary permissions reduces the potential impact of compromised credentials.

---

12. Result

The AWS IAM environment was successfully configured using the **Principle of Least Privilege.**

The **Admins, Developers, and Auditors** IAM groups were created with appropriate permissions. MFA was configured for administrative access, and an IAM Role was created for secure EC2-to-S3 access without hardcoded credentials.

---

13. Conclusion

This practical demonstrated the implementation of secure identity and access management in AWS. By using IAM groups, custom policies, MFA, and IAM Roles, access to AWS resources can be controlled according to user responsibilities.

The implementation demonstrates how the **Principle of Least Privilege** can reduce unnecessary permissions and improve the overall security of a cloud environment.